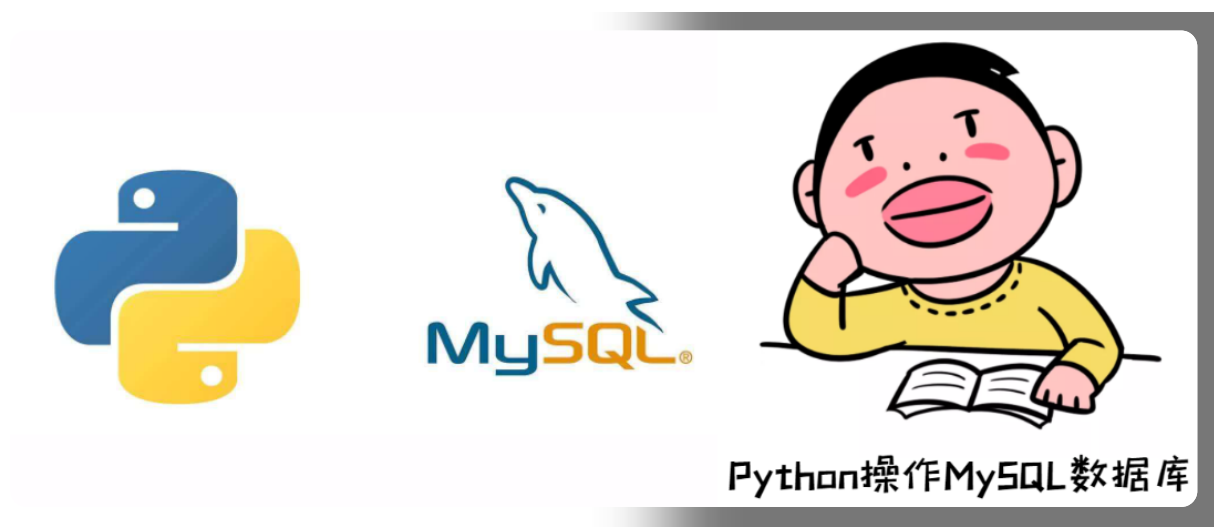


Python操作MySQL数据库



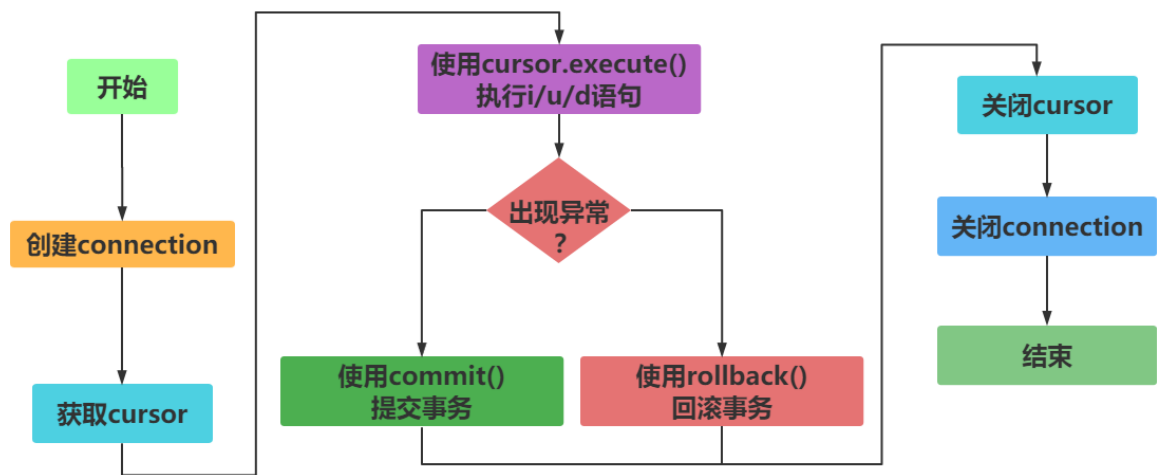
因为Python和MySQL是2套软件，所以在互相交互的时候需要一个“桥梁”。这个“桥梁”就是驱动！

- **mysqlldb**又叫MySQL-python
 - 优点：基于C开发的库，速度快
 - 缺点：在 Windows 平台安装非常不友好，经常出现失败的情况，多年不更新了，只兼容python2
- **mysqlclient**
 - 优点：基于C开发的库，速度快，兼容python3
 - 缺点：编译安装可能会导致报各种错误
- **pymysql**
 - 优点：纯 Python 实现的驱动，兼容python3，使用简单
 - 缺点：速度不如mysqlldb

安装pymysql

```
1 pip install pymysql
```

python操作数据流程



pymysql模块中的函数

连接数据库函数

connect函数：连接数据库，根据连接的数据库类型不同，该函数的参数也不同。connect函数返回Connection对象。

```

1 import pymysql
2 #获取连接
3 con =
  pymysql.connect(host="localhost",port=3306,user="root",password="root",db="test06",charset
    ='utf8')

```

获取游标

cursor方法：获取操作数据库的Cursor对象,包含了很多操作数据的方法。cursor方法属于Connection对象。

```

1 #创建cursor
2 cursor = connection.cursor()

```

执行sql语句

执行单条sql语句

```

1 execute(query,args=None)

```

函数作用：执行单条的sql语句，执行成功后返回受影响的行数

参数说明：

- query：要执行的sql语句，字符串类型
- args：可选的序列或映射，用于query的参数值。如果args为序列，query中必须使用%s做

批量执行SQL语句

```
1 executemany(query, args=None)
```

函数作用：批量执行sql语句，比如批量插入数据，执行成功后返回受影响的行数

参数说明：

- query：要执行的sql语句，字符串类型
- args：嵌套的序列或映射，用于query的参数值

提交事务

commit方法：在修改数据库后，需要调用该方法提交对数据库的修改。

```
1 #提交事务
2 con.commit()
```

回滚

rollback方法：如果修改数据库失败，一般需要调用该方法进行数据库回滚，也就是将数据库恢复成修改之前的样子。

```
1 #如果出现异常，回滚
2 con.rollback()
```

实时效果反馈

1. _____模块是在 Python3.x 版本中用于连接 MySQL 服务器的一个库。

 PyMySQL

B python

C MySQL

2. 在修改数据库后，需要调用_____提交对数据库的修改。

A `con.rollback()`

B `con.connect()`

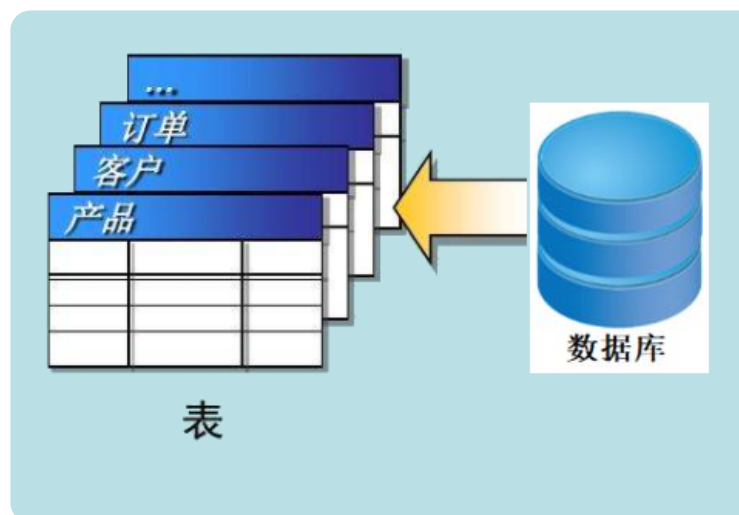
C `con.commit()`

D `con.close()`

答案

1=>A 2=>C

创建数据库和表



【示例】创建数据库

```
1 import pymysql
2
```

```
3 # 链接数据库
4 con =
    pymysql.connect(host='localhost',port=3306,u
        ser='root',passwd='root',charset='utf8')
5 # 获取一个和数据库交互的工具cursor
6 cursor = con.cursor()
7 # 编写SQL
8 sql = '''
9 CREATE DATABASE sxt
10     DEFAULT CHARACTER SET = 'utf8mb4';
11 '''
12 # 执行SQL
13 cursor.execute(sql)
14 # 关闭cursor
15 cursor.close()
16 # 关闭链接
17 con.close()
```

【示例】创建数据表

```
1 # pip install pymysql
2 import pymysql
3
4 # 链接数据库
5 con =
    pymysql.connect(host='localhost',port=3306,u
        ser='root',passwd='root',charset='utf8',db='
            sxt')
6 # 获取一个和数据库交互的工具cursor
7 cursor = con.cursor()
8 # 编写SQL
    sql = '''
```

```
10 CREATE TABLE t_user(  
11     id int NOT NULL PRIMARY KEY  
    AUTO_INCREMENT COMMENT 'Primary Key',  
12     uname VARCHAR(32) not null,  
13     age int(11) not null,  
14     sex VARCHAR(5)  
15 ) DEFAULT CHARSET UTF8;  
16 ''  
17 # 执行SQL  
18 cursor.execute(sql)  
19 # 关闭cursor  
20 cursor.close()  
21 # 关闭链接  
22 con.close()
```

实时效果反馈

1. Python连接MySQL数据库，调用_____返回Connection对象。

☒ A connection函数

☐ B connect函数

2. Python连接MySQL数据库，下划线处分别需要填写的代码是_____：

```
1 #导入模块
2 import pymysql
3 #创建连接
4 con = _____(host='localhost',
5                    user='root',
6                    password='root',
7                    db='db_test',
8                    charset='utf8')
9
10 #获取游标
11 cursor = _____
```

- A connect cursor()
- B pymysql.connection con.cursor()
- C con.cursor pymysql.connect()
- D pymysql.connect con.cursor()

答案

1=>B 2=>D

Python通过DML语句-增加数据

数据 插入修改删除



注意

在python的pymysql模块，如果需要操作DML语句,需要手动提交事务 `con.commit()`

【示例】插入数据

```
1 def add_one():
2     # 链接数据库
3     con =
4         pymysql.connect(host='localhost',port=3306,
5         user='root',passwd='root',db='sxt',charset='
6         utf8')
7         # 获取操作数据的对象 cursor
8         cursor = con.cursor()
9         # 编写SQL-DML
10        # sql = "INSERT INTO t_user VALUES
11        (0,'貂的蝉',18,'女');"
12        sql = "INSERT INTO t_user VALUES
13        (0,%s,%s,%s);"
14        args = ('刘备',22,'男')
15        # 执行SQL
16        cursor.execute(sql,args)
17        # 提交事务
```



```

13     con.commit()
14     # 关闭Cursor
15     cursor.close()
16     # 关闭链接
17     con.close()

```

【示例】插入多条数据

```

1  def add_many():
2      # 链接数据库
3      con =
        pymysql.connect(host='localhost',port=3306,
        user='root',passwd='root',db='sxt',charset='
        utf8')
4      # 获取操作数据的对象 cursor
5      cursor = con.cursor()
6      # 编写SQL-DML
7      sql = "INSERT INTO t_user VALUES
        (0,%s,%s,%s);"
8      args = (('孙权',23,'男'),('孙尚
        香',20,'女'))
9      # 执行SQL
10     cursor.executemany(sql,args)
11     # 提交事务
12     con.commit()
13     # 关闭Cursor
14     cursor.close()
15     # 关闭链接
16     con.close()

```

实时效果反馈

1. Python连接MySQL数据库，插入多条数据调用_____。

A `cursor.executemany()`

B `cursor.execute()`

C `cursor.executemore()`

2. Python连接MySQL数据库，插入多条数据下划线处需要填写的代码是____：

```
1 #插入sql语句
2 sql='insert into t_user(uname,sex,age)
  values(%s,%s,%s) '
3 args = _____
4 #执行sql语句
5 cursor.executemany(sql,args)
```

A `[('王五','man',24),('赵六','woman',27)]`

B `('王五','man',24),('赵六','woman',27)`

答案

1=>A 2=>A

Python操作DML语句-更新删除数据

数据 插入修改删除



注意

在python的pymysql模块，如果需要操作DML语句,需要手动提交事务 `con.commit()`

【示例】修改数据

```
1 #将id=2的年龄修改为18
2 sql='update t_user set age=%s where id=%s'
3 cursor.execute(sql,(18,2))
```

【示例】删除数据

```
1 #导入模块
2 import pymysql
3 #创建连接
4 con =
    pymysql.connect(host='localhost',user='root'
        ,password='root',db='db_test',charset='utf8'
    )
5
6 #创建游标对象cursor
7 cursor=con.cursor()
8 #删除id=2
9 sql='delete from t_user where id=2'
10 #执行sql
```

```
11 cursor.execute(sql)
12 #提交数据
13 con.commit()
14
```

实时效果反馈

1. Python连接MySQL数据库，修改数据下划线处需要填写的代码是_____：

```
1 #需求：将id=2的年龄修改为18
2 sql='update t_user set age=%s where id=%s'
3 #执行sql
4 cursor.execute(sql,_____)
```

A (2,18)

B (18,2)

C 2,18

D 18,2

2. Python连接MySQL数据库，删除数据下划线处需要填写的代码是_____：

```

1 #创建游标对象cursor
2 cursor=con.cursor()
3 #删除id=2
4 sql='delete from t_user where id=2'
5 #执行sql
6 cursor.execute(sql)
7 #提交事务
8 _____
9 print('删除成功')

```

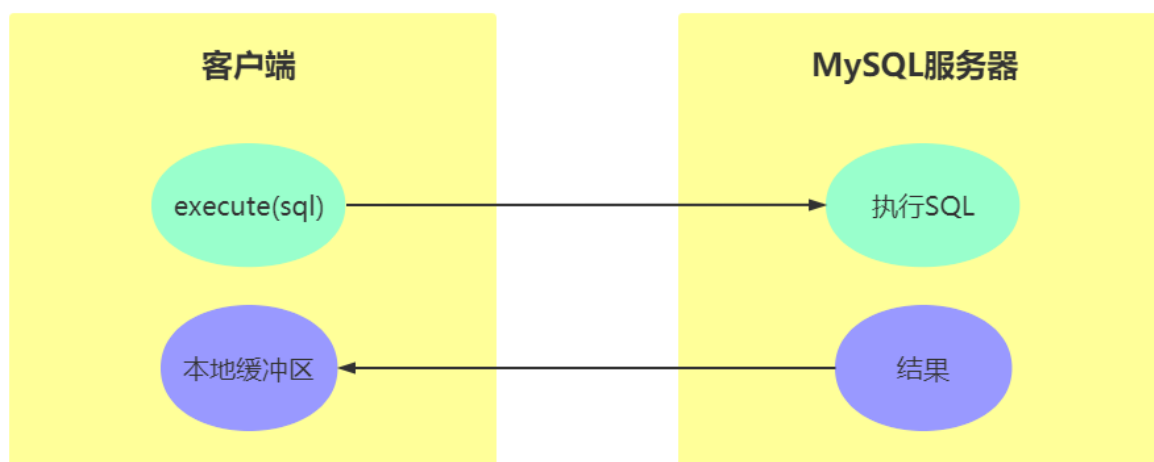
A con.commit()

B con.rollback()

答案

1=>B 2=>A

Python操作DQL-查询数据



Python查询Mysql使用 `fetchone()` 方法获取单条数据, 使用 `fetchall()` 方法获取多条数据。

- `fetchone()`: 该方法获取下一个查询结果集。结果集是一个对象

- fetchall(): 接收全部的返回结果行。
- fetchmany(num): 查询指定条数的记录。
- rowcount: 这是一个只读属性，并返回执行execute()方法后影响的行数。



【示例】查询数据

```

1  #导入模块
2  import pymysql
3  #创建连接
4  con =
    pymysql.connect(host='localhost',user='root'
        ,password='root',db='db_test',
5  charset='utf8')
6  #创建游标对象cursor
7  cursor=con.cursor()
8  sql='select * from t_user'
9  #执行sql
10 cursor.execute(sql)
11 #获取结果集
12 # result = cursor.fetchall()
13 # result = cursor.fetchmany(2)
14 result = cursor.fetchone()
15 print(result)

```

```
16 # print(cursor.rowcount)
```

实时效果反馈

1. Python查询Mysql数据库数据使用 _____方法获取单条数据, 使用fetchall()方法获取所有数据。

A `fetchcount()`

B `fetchall()`

C `fetchmany()`

D `fetchone()`

2. Python连接MySQL数据库, 查询数据下划线处需要填写的代码是_____:

```
1 sql='select * from t_user'  
2 #执行sql  
3 cursor.execute(sql)  
4 #获取所有查询结果集  
5 _____  
6 print(result)
```

A `result = cursor.fetchall()`

B `result = cursor.fetchone()`

答案

1=>D 2=>A

SQL注入查询漏洞



SQL注入即是指web应用程序对**用户输入数据的合法性没有判断或过滤不严**，攻击者可以在web应用程序中事先定义好的查询语句的**结尾上添加额外的SQL语句**，在管理员不知情的情况下实现非法操作，以此来实现欺骗数据库服务器执行**非授权的任意查询**，从而进一步得到相应的数据信息

测试参数

```
1 # login(uname,pwd)
2 login('吕小布" and 1=1#','')
```

【示例】无用写法

```
1 #编写sql
2 sql = 'SELECT id,uname,age,sex FROM t_user
   where uname="吕小布" and pwd="123";
3
4 # 执行
5 cursor.execute(sql)
6 # 获取数据
7 rs = cursor.fetchone()
8 print(rs)
```


【示例】被SQL注入版

```
1 #编写sql
2 sql = 'SELECT id,uname,age,sex FROM t_user
   where uname="' + uname + '" and pwd="' + pwd + '";'
3
4 sql = f'SELECT id,uname,age,sex FROM t_user
   where uname="{uname}" and pwd="{pwd}";'
5
6 # 执行
7 cursor.execute(sql)
8 # 获取数据
9 rs = cursor.fetchone()
10 print(rs)
```

【示例】安全版本

```
1 sql = 'SELECT id,uname,age,sex FROM t_user
   where uname=%s and pwd=%s;'
2 args = (uname,pwd)
3
4 # 执行
5 cursor.execute(sql,args)
6 # 获取数据
7 rs = cursor.fetchone()
8 print(rs)
```

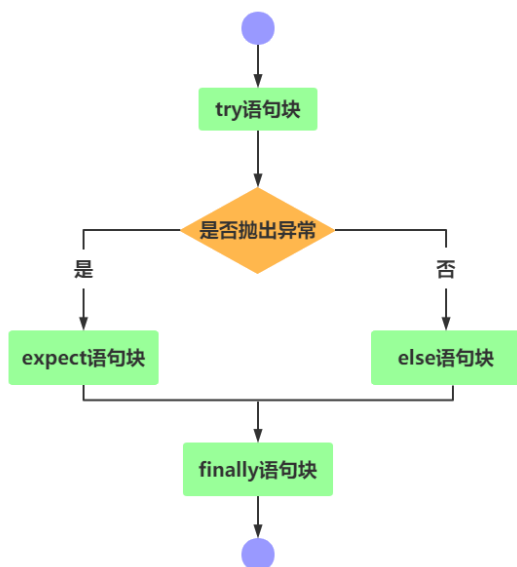
实时效果反馈**1. 关于SQL注入，说法错误的是？**

- A SQL注入因对用户输入参数没有检查或检查不严格造成
- B SQL注入因拼接SQL造成的
- C SQL注入是普遍问题，无法解决
- D SQL注入可以通过传参方式解决

答案

1=>C

添加异常处理



【示例】添加异常处理

```
1 import pymysql
2 connection=None
3 cursor=None
4 try:
5     ....
6 except Exception as e:
7     ....
8 finally:
9     if cursor:
10         cursor.close()
11     if connection:
12         cursor.close()
```

实时效果反馈

1. Python连接MySQL数据库，关闭数据连接代码应该放到

_____。

- ☒ A try语句块
- ☐ B except语句块
- ☐ C else语句块
- ☐ D finally语句块

答案

1=>D



问题

pymysql操作mysql，虽然简单，但每次都要链接数据库，获取游标，关闭游标，关闭链接。这些操作无技术含量，还要重复编写！！我们应该想法提高开发效率

解决方案

编写工具类，将公共的内容封装起来

- 获取链接，获取游标
- 关闭游标，关闭链接
- 合并DML方法
- 查询数据
 - 单条查询
 - 多条查询

```
1 import pymysql
2
3 class DBUtil:
4     config = {
```

```
5         'host': 'localhost',
6         'port': 3306,
7         'user': 'root',
8         'passwd': 'root',
9         'db': 'sxt',
10        'charset': 'utf8'
11    }
12    def __init__(self) -> None:
13        '''
14        获取链接
15        获取游标
16        '''
17        self.con =
pymysql.connect(**DBUtil.config)
18        self.cursor = self.con.cursor()
19
20    def close(self) ->None:
21        '''
22        关闭链接与游标
23        '''
24        if self.cursor:
25            self.cursor.close()
26        if self.con:
27            self.con.close()
28
29    def execute_dml(self,sql,*args):
30        '''
31        可以执行dml语句，用于数据的增加、删除、修改
32        '''
33        try:
34            # 执行SQL
35            self.cursor.execute(sql,args)
```

```
36         # 提交事务
37         self.con.commit()
38     except Exception as e:
39         print(e)
40         if self.con:
41             self.con.rollback()
42     finally:
43         self.close()
44
45     def query_one(self, sql, *args):
46         '''
47         获取一条数据
48         '''
49         try:
50             # 执行SQL
51             self.cursor.execute(sql, args)
52             # 获取结果
53             rs = self.cursor.fetchone()
54             # 返回数据
55             return rs
56         except Exception as e:
57             print(e)
58         finally:
59             self.close()
60
61     def query_all(self, sql, *args):
62         '''
63         获取所有数据
64         '''
65         try:
66             # 执行SQL
67             self.cursor.execute(sql, args)
```

```
68         # 获取结果,并返回数据
69         return self.cursor.fetchall()
70     except Exception as e:
71         print(e)
72     finally:
73         self.close()
74
75
76 if __name__ == '__main__':
77     db = DBUtil()
78     sql = 'select * from t_user where sex =
79         %s'
80     # print(db.query_one(sql,3))
81     print(db.query_all(sql,'男'))
```